

Battery Rush Arena

게임네트워크기초 중간 프로젝트

Unity 6000.3

.NET 10 / C#

MySQL 8.x

TCP Socket

2인 실시간 배틀 게임 · 권위 서버 아키텍처 · MySQL 전적 저장

목차

01

프로젝트 개요

게임 소개 & 기술 스택

03

서버 컴포넌트

C# .NET 10 서버 모듈 10개

05

메시지 타입

C→S 8개 / S→C 16개

07

게임 로직

이동·배터리·함정·SlowShot

02

전체 시스템 구조

Unity ↔ Server ↔ MySQL

04

프로토콜 설계

Length-Prefix + JSON 설계 결정

06

룸 상태 머신

Lobby → ResultsReady 전이

08

DB 스키마 & 저장 흐름

MySQL 테이블 & Transaction

프로젝트 개요

게임 소개

- ▶ 2인 실시간 배틀 아레나 게임
- ▶ 배터리를 먹어 점수를 쌓고, 함정·SlowShot으로 상대 방해
- ▶ 120초 매치 — 종료 시 MySQL에 전적 자동 저장

아키텍처 특징

- ▶ 권위 서버(Authoritative): 게임 로직 100% 서버 처리
- ▶ Unity 클라이언트 — 입력 전송 + 렌더링 전담 (경량)
- ▶ MySQL 직접 접속 없음 (서버 경유 전용)

프로토콜

- ▶ TCP 소켓 + Length-Prefix + UTF-8 JSON
- ▶ 프로토콜 버전: bra-spike-v1

Unity 6000.3.10f1

클라이언트 렌더링 & 입력 처리

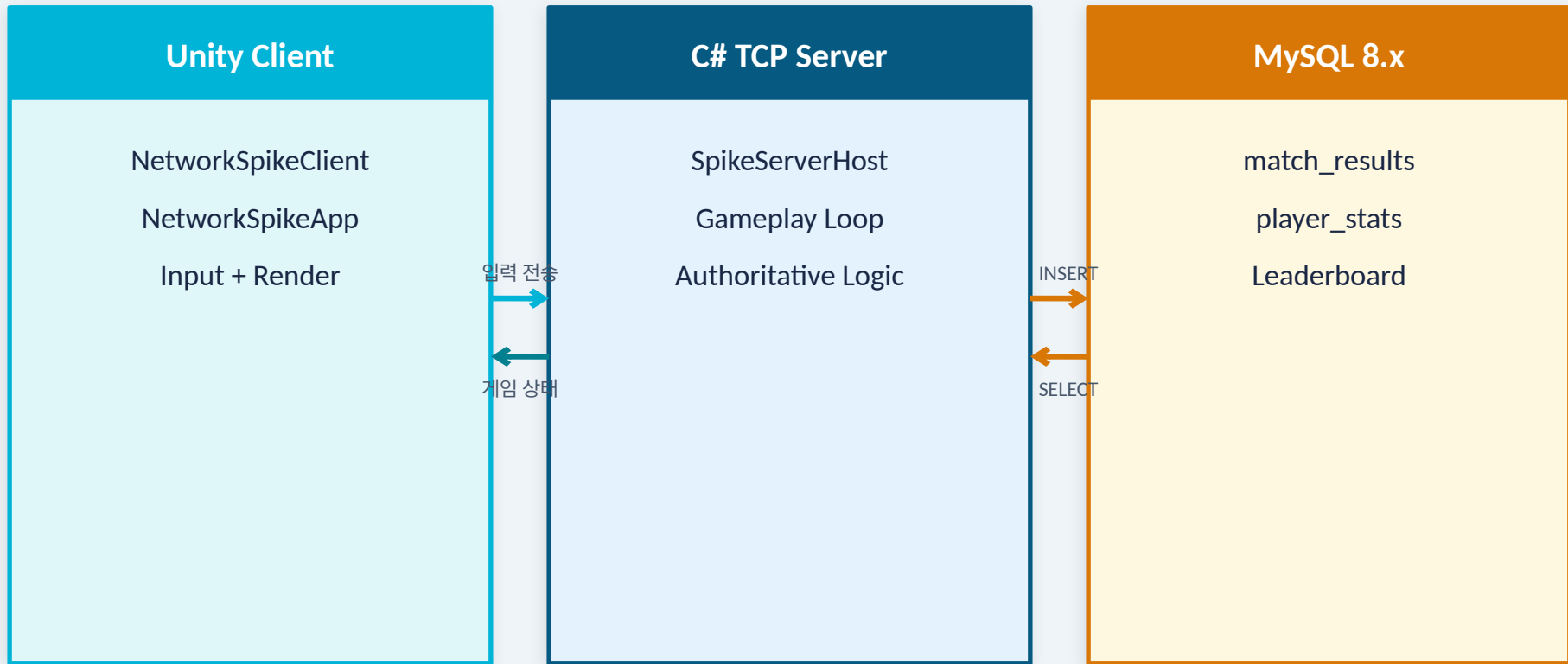
.NET 10 / C#

TCP 서버 · 게임 로직 · DB 연결 관리

MySQL 8.x

전적 저장 (match_results·player_stats)

전체 시스템 구조



서버 컴포넌트 구조

Program.cs

진입점 · SpikeServerHost 생성 및 RunAsync()

ClientSession.cs

연결 클라이언트 상태 (SessionId, LastSeenUtc, LastTick)

SpikeRoom.cs

룸 상태 · PlayerEffectState · SpikeVec2 · 게임 로직

ProtocolMessages.cs

ClientMessage / ServerMessage DTO · ProtocolJson 설정

SpikeServerConfig.cs

포트·타임아웃·게임 파라미터 상수 모음

SpikeServerHost.cs

TCP Accept 루프 · 메시지 switch 디스패치 · 게임플레이 루프

RoomRegistry.cs

인메모리 룸 관리 · SyncRoot 락으로 스레드 안전

LengthPrefixedProtocol.cs

4-byte BigEndian 길이 헤더 + UTF-8 JSON · SemaphoreSlim

MySQLPersistenceService.cs

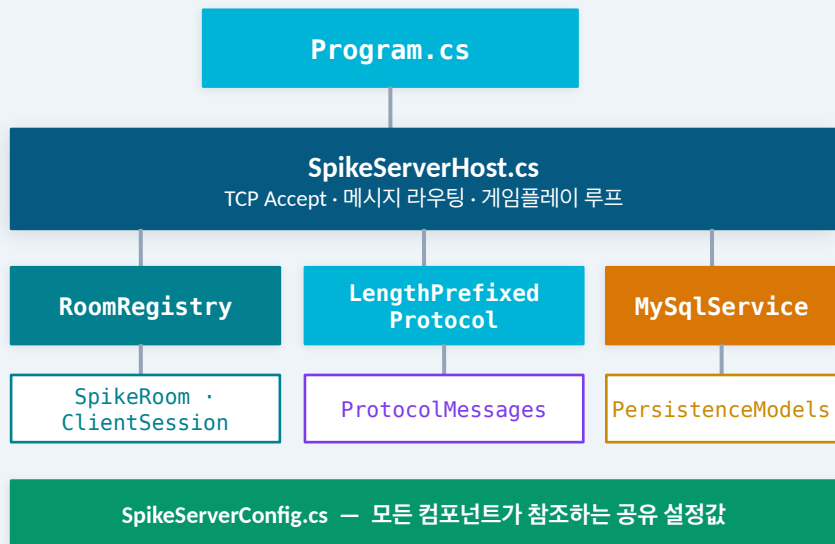
비동기 INSERT + UPSERT · 최대 3회 재시도 · QueryTop10

PersistenceModels.cs

MatchResultPayload · LeaderboardRow · PersistenceAttemptResult

컴포넌트 통신 흐름도

의존 관계



패킷 처리 흐름 (input_frame 예시)

- 1 TCP 바이트 수신**
SpikeServerHost · HandleClientAsync()
- 2 역직렬화**
LengthPrefixedProtocol · ReadAsync<ClientMessage>()
- 3 메시지 라우팅**
message.Type → HandleInputFrameAsync()
- 4 게임 로직 적용**
RoomRegistry + SpikeRoom 위치·효과 갱신
- 5 브로드캐스트**
LengthPrefixedProtocol · WriteAsync → 전체 클라이언트

프로토콜 설계

패킷 구조

Length	JSON Payload
4 bytes · BigEndian int32	UTF-8 string · { "type": "...", ... }

```
// 예시
{
  "type": "input_frame",
  "dx": 1, "dy": 0
}
```

설계 결정

SemaphoreSlim(1,1) per stream

스트림당 뮤텍스로 동시 쓰기 시 패킷 인터리빙 방지. WriteAsync 직렬화 보장.

CaseInsensitive JSON 파싱

Unity ↔ C# 서버 간 필드명 대소문자 불일치 허용. ProtocolJson 설정으로 강건성 향상.

WriteIndented = false

프로덕션 트래픽 최소화. 디버그 시에만 pretty-print 전환. 패킷 크기 절감.

메시지 타입 목록

Client → Server (8 types)

hello	핸드셰이크 · 버전 검증
create_room	룸 생성
join_room	룸 참가
leave_room	룸 탈퇴
ready_state	준비 상태 토글
start_match	매치 시작 (호스트 전용)
heartbeat	연결 유지 (2초 간격)
input_frame	플레이어 입력 (dx, dy, useSlowShot)

Server → Client (16 types)

hello_accepted	핸드셰이크 성공
hello_rejected	버전 불일치 / 이름 오류
heartbeat_ack	연결 유지 확인
room_joined	룸 참가/생성 성공
room_left	룸 탈퇴 처리
room_state_changed	룸 상태 변경
room_snapshot	룸 전체 스냅샷
room_listings_updated	룸 목록 갱신
countdown_tick	카운트다운 (3, 2, 1)
active_tick	매 프레임 게임 상태
battery_respawned	배터리 리스폰 위치
effect_state_changed	플레이어 효과 변경
input_frame_ack	입력 수신 확인
input_frame_ignored	중복/무효 입력 무시
session_stale	30초 타임아웃 경고
error	오류 응답

룸 상태 머신 (Room State Machine)



Lobby

플레이어 대기
룸 목록에 노출

Active

서버 tick 루프
배터리·함정 갱신

Saving

MySQL Transaction
INSERT + UPSERT

ResultsReady

리더보드 전송
클라이언트 결과 화면

게임 로직 ① 이동 & 배터리

이동 시스템

PlayerStepDistance 0.75 unit/frame

ArenaHalfExtent ±7.25 units

SpawnPoint A (-5.1, 0)

SpawnPoint B (5.1, 0)

StaleTimeout 30초

Heartbeat 2초 간격

배터리 시스템

ActiveBatteryCount 3개 동시

PickupRadius 0.65 units

RespawnDelay 3초 후 재생성

점수 배터리 1개 = +1점

배치 서버 랜덤 생성

알림 battery_respawned

게임 로직 ② 함정 & SlowShot

함정 시스템 (Trap)

ActiveTrapCount 4개 동시 배치

TriggerRadius 0.65 units

SpeedMult 0.70× 속도 감소

EffectDuration 0.75초

알림 effect_state_changed

효과 상태 SlowFromTrap

SlowShot 시스템

SlowShotRange 3.5 units

SpeedMult 0.65× 속도 감소

EffectDuration 1.25초

Cooldown 4초

발사 조건 dot product ≥ 0.25

효과 상태 SlowFromShot

Input Frame 처리 흐름

Unity Client

① 키 입력 감지

SendInputFrameAsync(dx, dy, useSlowShot)

② 패킷 전송

LengthPrefixed Protocol · WriteAsync

⑧ UI 갱신

MainThread Dispatch → 위치 업데이트

SpikeServerHost

③ 패킷 수신

ReadMessageAsync → switch(type)

④ 프레임 검증

lastProcessedTick + 1 확인

⑥ ACK 전송

input_frame_ack / input_frame_ignored

SpikeRoom

⑤ 게임 로직 처리

ApplyInput → 이동·배터리·함정 처리

⑦ 상태 브로드캐스트

active_tick → 전체 클라이언트

DB 스키마 설계

match_results

match_id	BIGINT AUTO_INCREMENT	PK — 자동 증가
room_id	VARCHAR(64)	룸 식별자
ended_at_utc	DATETIME	매치 종료 시각
end_reason	VARCHAR(32)	timeout / disconnect
winner_player_name	VARCHAR(64)	승자 이름
player_count	TINYINT	참가 인원
player_a / b name	VARCHAR(64)	각 플레이어 이름
player_a / b score	INT	각 플레이어 점수
raw_payload_json	JSON	전체 결과 JSON

player_stats

player_name	VARCHAR(64)	PK
wins	INT	총 승리
draws	INT	총 무승부
losses	INT	총 패배
best_score	INT	최고 점수
total_matches	INT	총 게임 수
last_played_at	DATETIME	최근 게임

INSERT match_results +
UPSERT player_stats
하나의 Transaction으로 원자 처리

MySQL 저장 흐름

1

매치 종료

Active → Ended: MarkRoomEnded()



2

Payload 생성

BuildMatchResultPayload() → MatchResultPayload 객체



3

State: Saving

상태 Saving으로 전이 · 클라이언트 broadcast



4

DB Transaction

BEGIN → INSERT match_results → UPSERT player_stats × N



5

COMMIT / 재시도

성공 시 COMMIT, 실패 시 최대 3회 retry 후 롤백



6

Leaderboard 조회

QueryTop10() → SELECT ORDER BY best_score DESC LIMIT 10

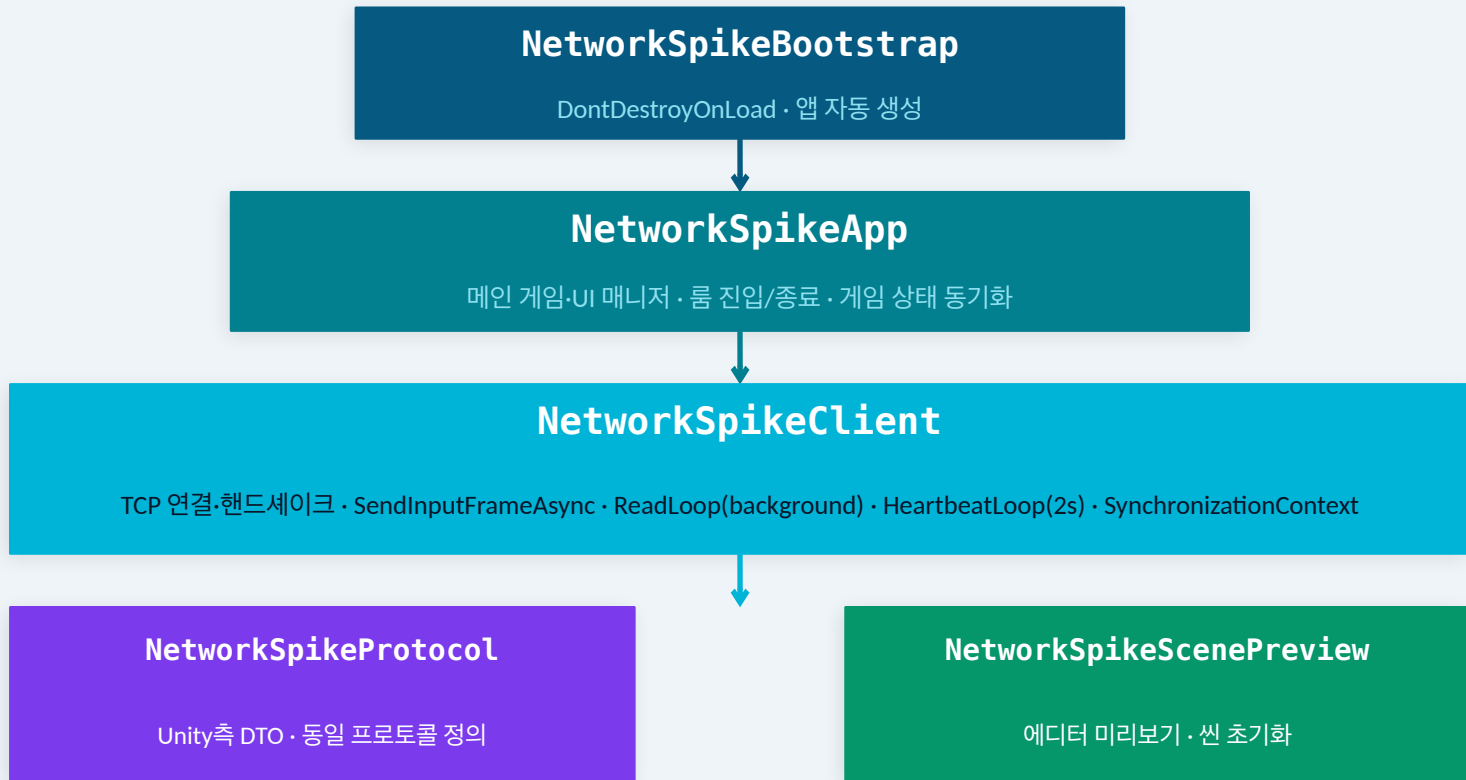


7

State: ResultsReady

BroadcastRoomAsync(leaderboard) → 클라이언트 결과 화면

Unity 클라이언트 구조



정리 & 데모 동선

핵심 요약

- 권위 서버로 게임 로직 100% 서버 처리
- TCP + Length-Prefix JSON으로 신뢰성 있는 통신
- 룸 상태 머신 6단계 (Lobby → ResultsReady)
- MySQL Transaction으로 원자적 전적 저장 (3회 재시도)
- HeartbeatLoop + StaleTimeout으로 연결 관리
- Unity는 입력 전송 + 렌더링만 담당 (경량 클라이언트)

데모 동선

- 1 서버 시작 dotnet run (Port 7777)
- 2 Unity 실행 A Player A 접속 · 룸 생성
- 3 2번째 클라이언트 Player B 참가 · Ready 설정
- 4 매치 시작 호스트 Start → Countdown → Active
- 5 게임플레이 이동·배터리·함정·SlowShot 시연
- 6 DB 확인 MySQL match_results + player_stats